

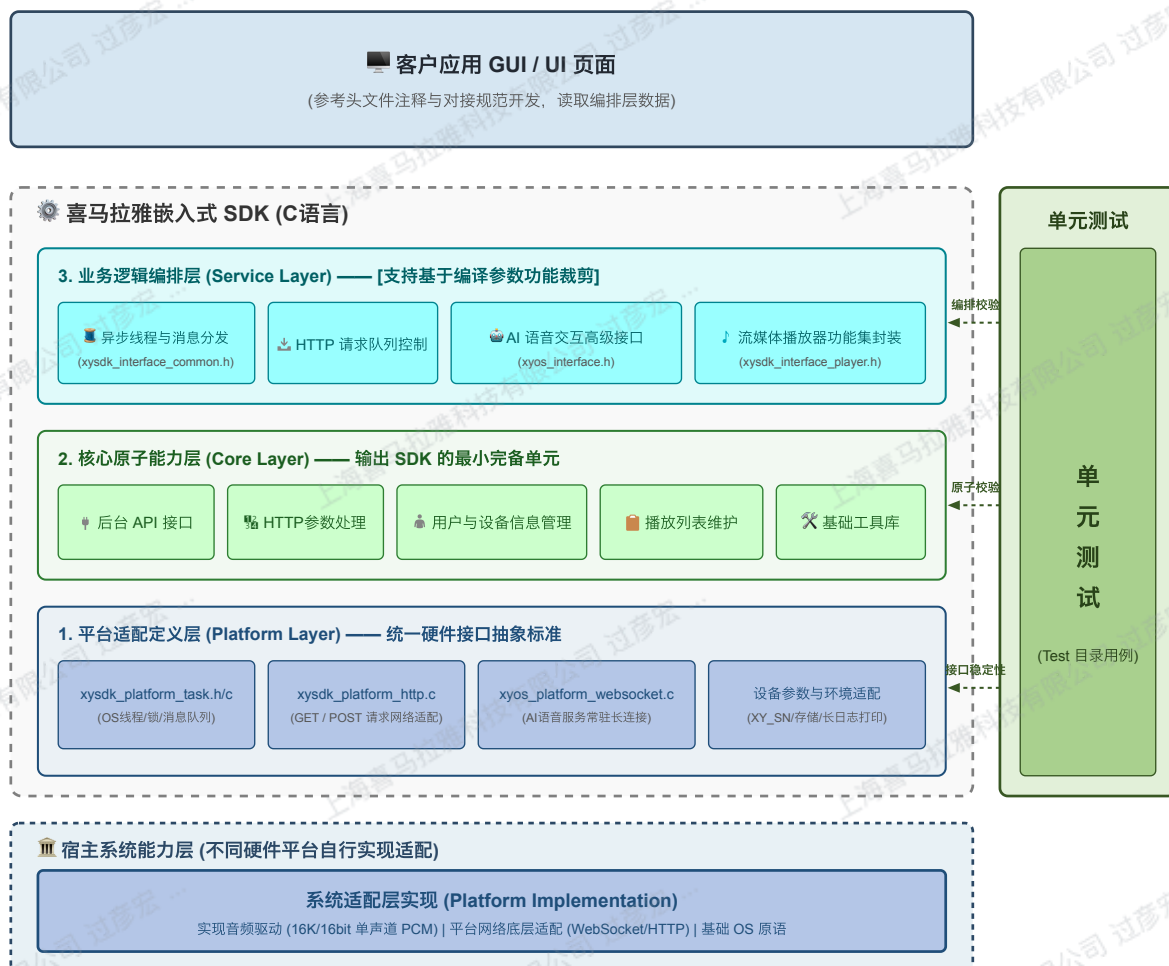
喜马拉雅嵌入式SDK（C语言）详解

v2.2

一、SDK介绍

1. SDK框架

嵌入式SDK框架分三层设计（如下图），即核心原子能力（core）、业务逻辑编排（service）和平台适配（platform）定义。并针对这三层架构分别设计了测试用例，要来测试接口的稳定性。

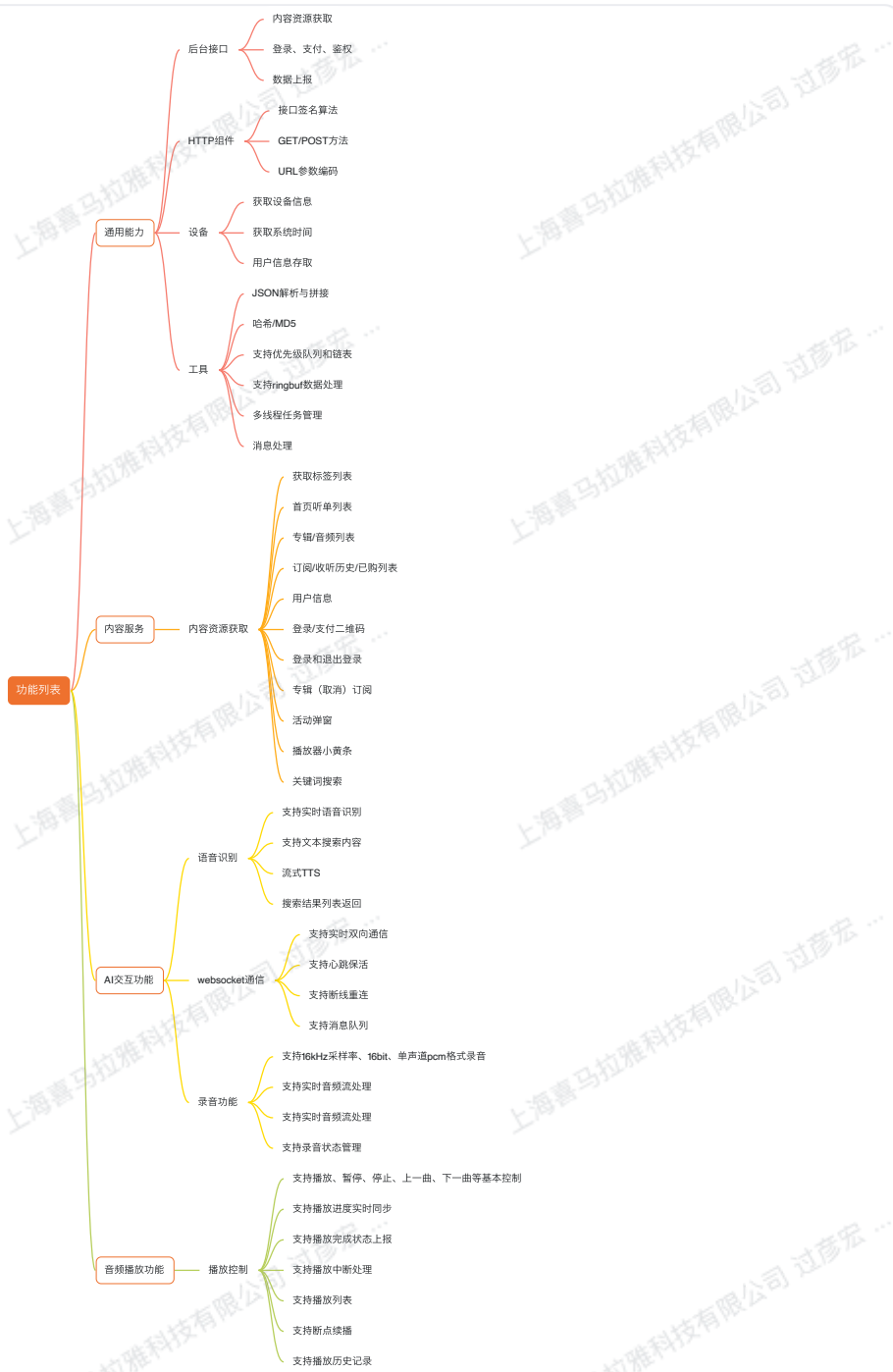


核心原子能力主要是处理核心功能模块，是输出sdk的最小单元，包含后台API接口、http参数处理、用户（设备）信息管理、播放列表维护和基础组件工具库；

业务逻辑编排是在核心层的基础上做一些逻辑处理，对一些UI的逻辑显示数据做一些接口的封装，主要包含线程的创建、消息的分发与处理、http请求队列、内容接口封装、播放器API封装及用户信息接口封装。业务逻辑编排由客户自行决定是否选用，选用后也可根据功能需求用编译参数做功能裁剪。

平台适配定义是针对不同平台定义了一些sdk中需要用到的接口，客户拿到SDK后需要根据系统来适配该层定义的接口，完成SDK与系统能力的适配。主要包含**录音功能实现、websocket连接、http接口适配、线程\锁\消息\信号量等接口适配、平台参数获取、数据存取、日志打印和流媒体播放接口实现**。目前提供的sdk是基于Linux平台实现的，不同平台需要自行适配。

2. SDK功能列表



二、接入条件

嵌入式设备接入sdk可定制功能，不同功能条件也不一样，具体如下（不含本地UI图片资源）

	内容	AI Agent	内容+AI Agent
ROM(code分区)	80KB	50KB	110KB
RAM（运存）	70KB	40KB	100KB
需支持基础能力	MP3流播放 \HTTP\pthread d	MP3流播放 \websocket\ HTTP\pthread	MP3流播放\websocket\ HTTP\pthread

三、接入流程

客户接入喜马SDK需要按照以下流程执行：

1. 确认开发需求，只接核心Core还是完整SDK对接，以及SDK中的AI和播放器功能是否需要；
2. 客户提供交叉编译工具链、开发环境及接收邮箱；
3. 喜马搭建开发环境并根据需求编译sdk，将核心代码封装成库，同UI切图一起通过邮箱提供给客户；
4. 客户收到sdk后，实现platform目录的适配函数，platform适配后可用test目录对接口做测试；
5. 适配完平台后，参考头文件注释和对接文档开发UI页面；
6. 开发过程中有新需求或碰到问题，及时联系喜马；
7. 开发完成后，喜马需要验收（功能和UI）；

四、平台适配

平台适配定义的函数在platform目录，需要客户根据平台来实现（注意头文件中的函数是SDK中调用的，这些函数原型不能修改），主要分为以下几部分：

1. task相关接口适配

- 适配xysdk_platform_task.h中的类型定义
- 适配xysdk_platform_task.c中线程、锁、信号量和消息相关接口

2. http接口适配

- 适配xysdk_platform_http.c中http的get和post函数，需要同步请求。
- 配置xysdk_platform_http.h中XY_RESPONSE_TEXT_SIZE（http返回数据长度）值，建议20KB。

3. AI相关接口适配（支持AI功能时适配）

- 适配xyos_platform_record.c中录音相关函数，目前sdk支持的录音格式是16K、16bit单声道pcm格式
- 适配xyos_platform_websocket.c中的websocket协议相关接口

4. 设备信息适配

- 适配xysdk_platform.c中的时间戳、延时、内存申请等函数
- 儿童SDK需要打开xysdk_platform.h中的XYSDK_SUPPOORT_CHILD_FUNC宏
- 配置xysdk_platform.h中的XY_CLOUD_KEY、XY_CLOUD_SECRET宏定义的参数（喜马拉雅提供）
- xysdk_platform.h中xy_msg_player_status_e（播放消息）、xy_msg_ai_resp_type_e（AI消息）、xy_msg_http_type_e（http消息）和xy_msg_sys_type_e（系统消息）四个枚举是对应消息值（每个值都是唯一的，不能重复），用作通信和回调的消息ID
- 适配xysdk_log.h中关于打印的宏定义文件
- 适配xysdk_platform.c中长数据的打印函数（xy_print_long_data (...)）

5. 流媒体播放适配（支持播放器功能时适配，并需要回调播放状态xy_stream_status_e）

- 适配xysdk_platform_stream_play.c中流媒体播放相关函数。如果没有播放url的接口，可使用sdk中xysdk_stream_rbuf.h的rbuf接口做数据缓存(可以参考platform_asr目录下的代码)
- 返回的音频资源大部分为mp3格式，小部分为m4a格式。无pcm/opus格式
- 返回的url只能流式播放，绝不允许缓存到本地，违者后果自负。

6. 数据存取

- xysdk_interface.c（demo）中实现UI所需内容数据的存取（ram）和回调事件处理，供参考，可自行修改
- 适配xysdk_platform.c中用户信息存取接口（文件系统或DB数据库）

五、API接口说明

对外的API接口在以下几个头文件中，接口的具体说明可查看相应头文件

1. 公共函数接口xysdk_interface_common.h

- 应用是否在运行中

bool xysdk_interface_xm_app_is_running(void);

- 发送消息给主线程

int xysdk_interface_main_task_msg_send(main_msg_type_e type, xy_msg_http_type_e sub_type, void *data, uint32_t data_len);

- 创建主线程

int xysdk_interface_main_task_create(http_data_cb http_cb, ai_data_cb ai_cb);

- 销毁主线程

int xysdk_interface_main_task_destroy(void);

2. AI函数接口xyos_interface.h

- 给AI线程发送消息

int xyos_interface_ai_task_msg_send(xyai_msg_type_e type, void *data, uint32_t data_len);

- tts状态上报

void xyos_interface_ai_tts_status_upload(xyai_tts_status_e status, char *tts_token);

- AI播放状态上报, 用作断点记录

void xyos_interface_ai_player_status_upload(xyai_player_data_t *play_info);

- 启动AI线程

int xyos_interface_ai_func_init(ai_data_cb ai_cb);

- 销毁AI线程

int xyos_interface_ai_func_deinit(void);

- 保存AI连接相关信息

int xyos_interface_ai_save_os_data(xy_get_os_info_resp_model *data);

- 连接AI服务器

int xyos_interface_ai_connect_server(void);

3. 内容接口函数xysdk_interface_content.h

- 判断用户是否登录

bool xysdk_interface_get_user_is_login(void);

- 获取用户登录数据

xy_user_info_model *xysdk_interface_get_user_info(void);

- 保存用户登录数据(仅用于无屏设备)

int xysdk_interface_save_user_info(char *user_data, uint32_t user_data_len);

- 退出登录时清除用户信息(仅用于无屏设备)

int xysdk_interface_clear_user_info(void);

- 判断用户是否是vip

bool xysdk_interface_get_user_is_vip(void);

- 获取用户vip信息

xy_user_vip_info *xysdk_interface_get_user_vip_info(void);

- http请求获取分类列表

void xysdk_interface_request_tag_list(void);

- http请求首页信息

void xysdk_interface_request_homepage_info(char *tag_id);

- http请求播放页小黄条广告信息

void xysdk_interface_request_w_xiaohuangtiao_info(void);

- http请求弹窗信息

void xysdk_interface_request_popup_screen_info(uint8_t need_qr_img);

- http通过听单id请求听单信息

void xysdk_interface_request_listen_info(char *id, uint8_t page);

- http通过专辑id获取音频列表

void xysdk_interface_request_tracklist_info_by_album_id(xy_get_track_list_req_model *req_param);

- http请求登录支付二维码信息

void xysdk_interface_request_qrcode_info(xy_get_qrcode_req_model *qrcode_req_model);

- http请求当前二维码状态

void xysdk_interface_request_qrcode_status(char *qrcode_id);

- http请求用户vip信息

void xysdk_interface_request_vip_info(void);

- http请求退出登录

void xysdk_interface_request_logout(void);

- http请求个人中心的专辑列表（包括订阅、已购和收听历史）

void xysdk_interface_request_user_album_list(uint8_t type, uint8_t page);

- **http请求专辑的订阅状态**

```
void xysdk_interface_request_album_subscribe_status(char *album_id);
```

- **http请求专辑的订阅或取消订阅控制**

```
void xysdk_interface_request_ctrl_album_subscribe(subscribe_param_t *param);
```

- **http获取ai推荐的引导词**

```
void xysdk_interface_request_ai_goodcase_info(void);
```

- **http获取ai的连接地址等鉴权信息**

```
void xysdk_interface_request_ai_auth_info(void);
```

- **http获取音频是否已购**

```
void xysdk_interface_request_track_bought_status(char *album_id, char *track_id);
```

- **http获取专辑播放记录**

```
void xysdk_interface_request_album_history_record(char *album_id);
```

- **http批量获取专辑信息**

```
void xysdk_interface_request_batch_album_info(char *ids);
```

- **http批量获取声音信息**

```
void xysdk_interface_request_batch_track_info(char *ids);
```

- **http通过关键词搜索专辑信息**

```
void xysdk_interface_request_album_search(char *query_text, uint16_t text_len);
```

4. 播放器接口函数xysdk_interface_player.h

- **启动播放器**

```
int xysdk_interface_stream_player_start(track_info_t *track_info);
```

- **播放控制**

```
int xysdk_interface_stream_player_ctrl(xy_stream_ctrl_cmd_e cmd);
```

- **判断播放器是否启动**

```
bool xysdk_interface_player_is_running(void);
```

- **获取当前播放进度**

```
uint16_t xysdk_interface_get_play_progress(void);
```

- **获取是否在播放中**

```
bool xysdk_interface_get_player_status(void);
```

- **获取当前播放的音频信息**

```
track_info_t *xysdk_interface_get_cur_playing_info(void);
```

- 获取播放历史记录(本地缓存的上一次播放)

track_info_t *xysdk_interface_get_play_history_record(void);

- 重新播放

int xysdk_interface_stream_player_restart(uint16_t seek_time);

- 获取当前播放列表

xy_get_source_list_resp_model *xysdk_interface_get_playlist(void);

- 挂起喜马拉雅播放器

void xysdk_interface_suspend_player(void);

- 播放器初始化（内存申请和线程创建）

int xysdk_interface_player_init(void);

- 播放器注销

void xysdk_interface_player_deinit(void);

- 启动TTS播放

int xysdk_interface_tts_player_start(char *url, uint16_t len);

- 停止TTS播放

int xysdk_interface_tts_player_stop(void);

- 判断是否在播放tts

bool xysdk_interface_is_tts_playing(void);

- 设置蓝牙状态

void xysdk_interface_set_bt_state(int state);

- 获取播放循环模式

xy_play_mode_e xysdk_interface_get_play_mode(void);

- 设置播放循环模式

void xysdk_interface_set_play_mode(xy_play_mode_e play_mode);

- 获取播放列表排序模式

xy_playlist_sort_e xysdk_interface_get_playlist_sort_mode(void);

- 设置播放列表排序模式

void xysdk_interface_set_playlist_sort_mode(xy_playlist_sort_e sort_mode);

- 获取播放倍速

xy_play_rate_e xysdk_interface_get_play_rate(void);

- 设置播放倍速

void xysdk_interface_set_play_rate(xy_play_rate_e play_rate);

5. 平台适配接口函数（demo）xysdk_interface.h

该头文件是针对平台来开发的，sdk中的接口供参考。其中以下几个接口需要重点关注下

- 创建喜马拉雅常驻（主）线程（函数里面设置了http回调和ai回调，数据解析参照这两个回调函数来实现）

```
int xysdk_interface_permanent_thread_create(send_msg_cb send_msg_cb, char *device_id);
```

- 应用初始化

```
int xysdk_interface_sevice_init(void);
```

- 应用退出

```
void xysdk_interface_sevice_deinit(void);
```

- 销毁喜马拉雅常驻（主）线程

```
int xysdk_interface_permanent_thread_delete(void);
```

- 获取AI服务器连接url并启动websocket连接

```
int xysdk_interface_ws_connect_start(bool force_update);
```

- AI录音操作指令

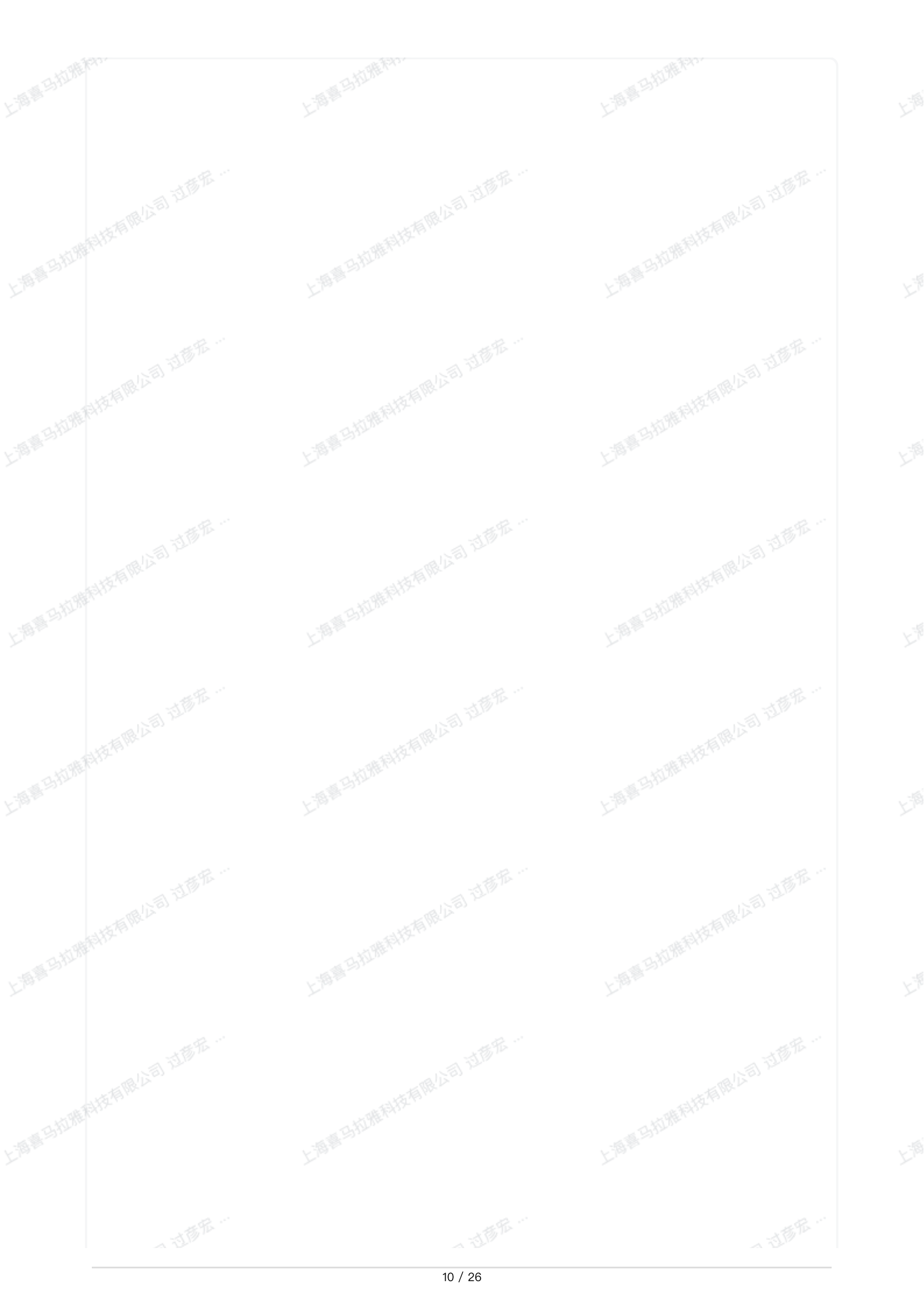
```
void xysdk_interface_recorder_cmd(uint8_t cmd);
```

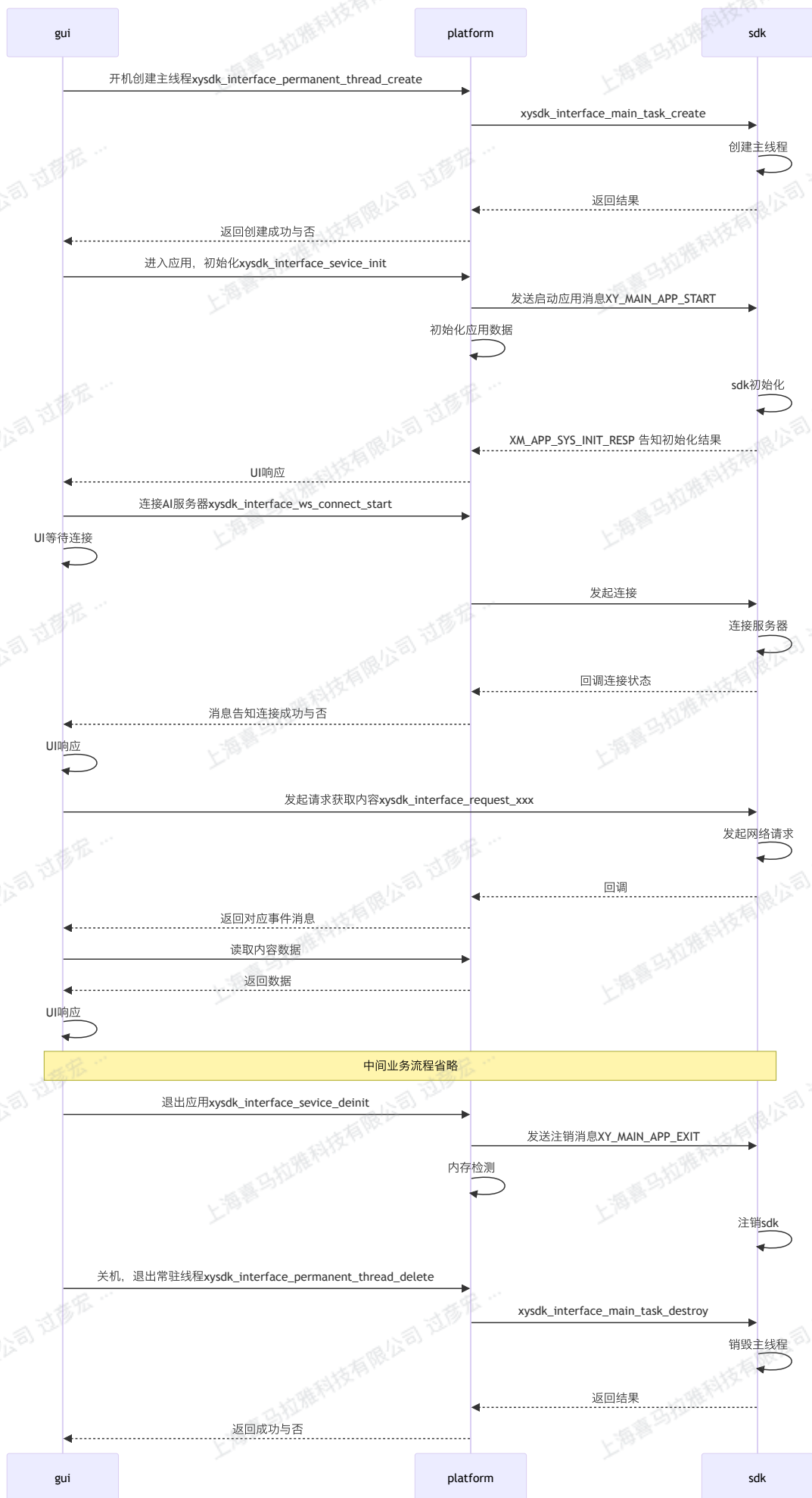
六、对接时序图

对接SDK主要包含AI连接与通信、http数据请求和播放器控制等核心流程，具体时序如下。其中不带屏设备操作流程详见5。

1. 应用操作时序图：

连接AI服务器流程仅在需要时调用

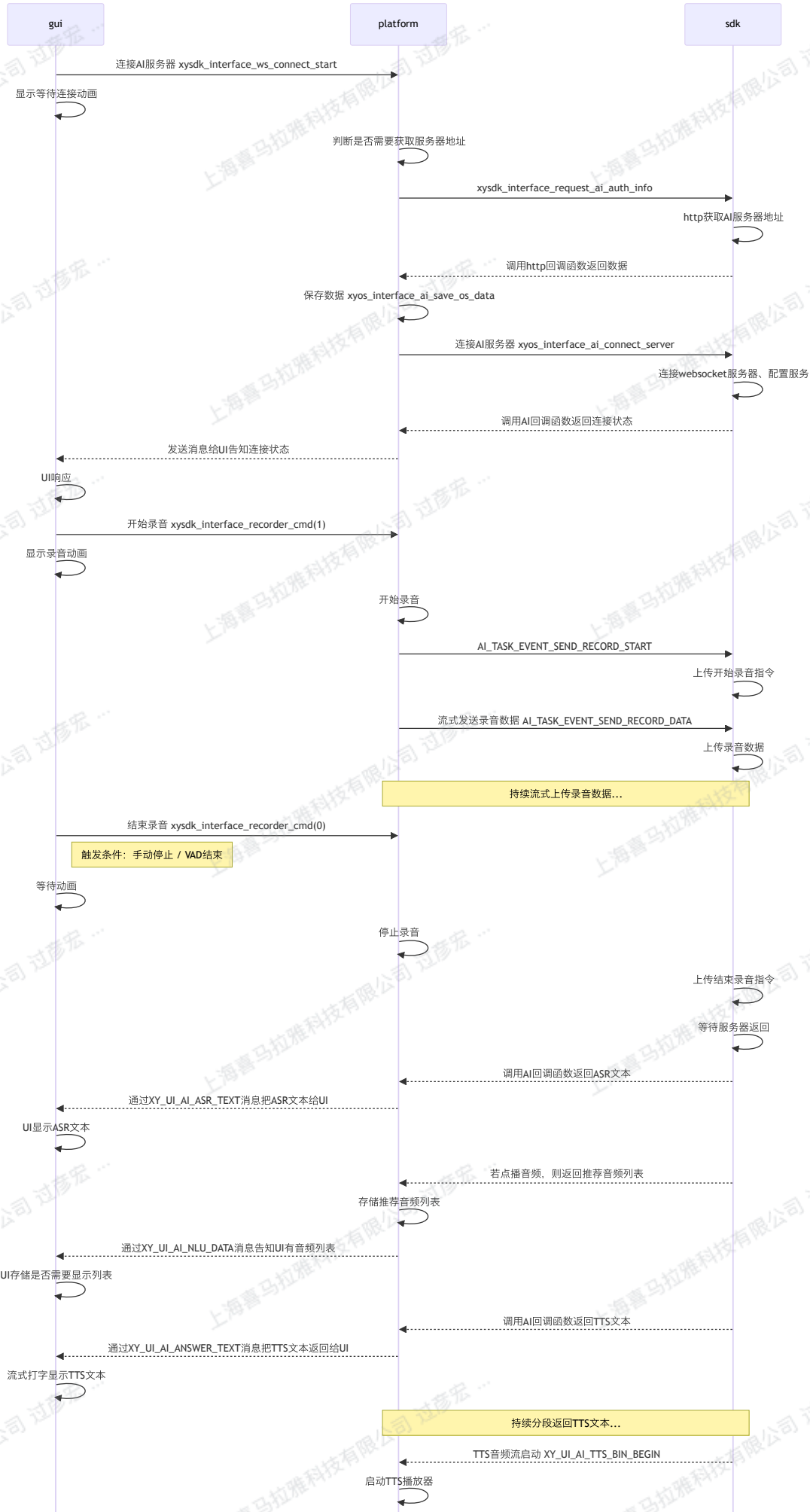


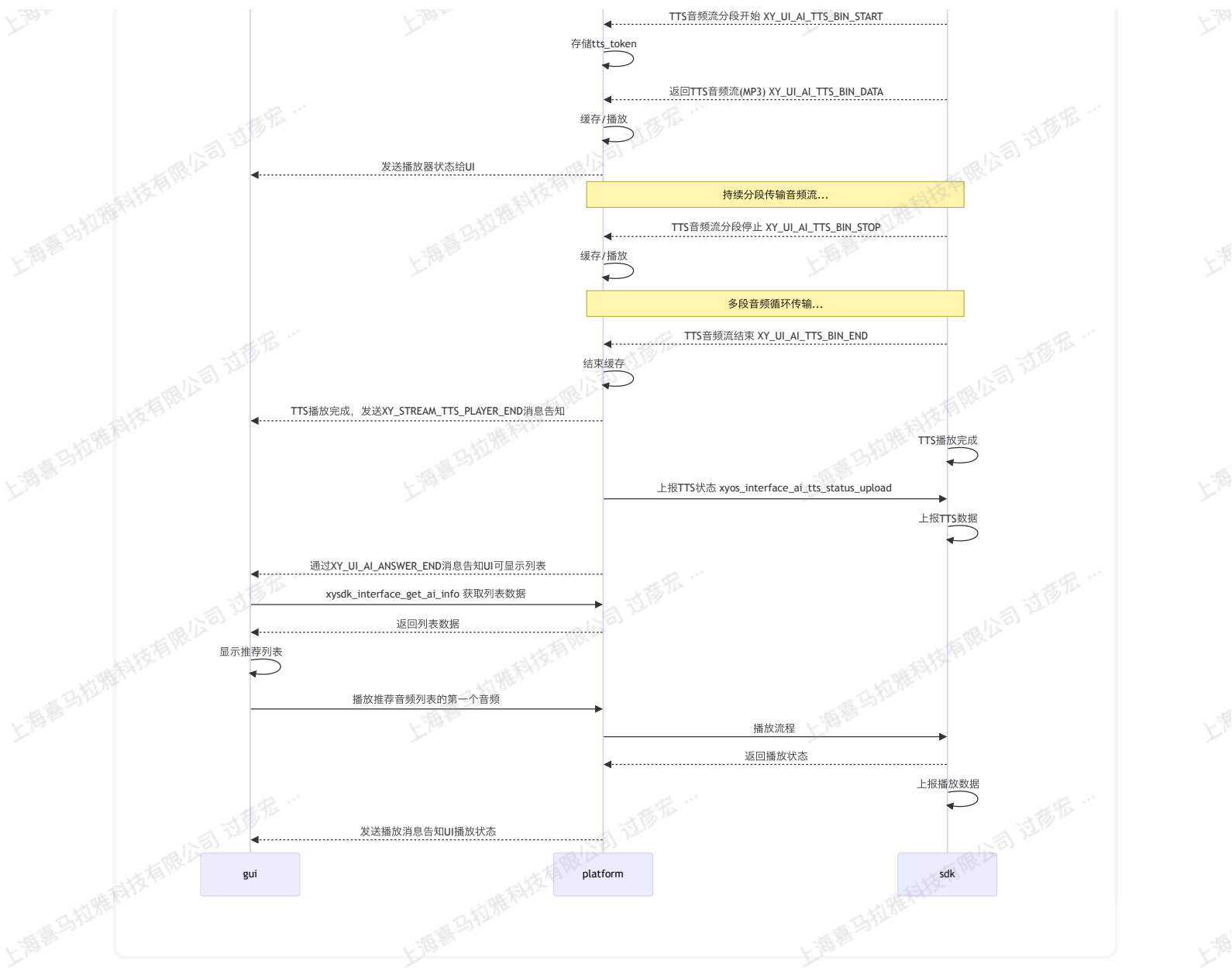


2. AI功能时序图：



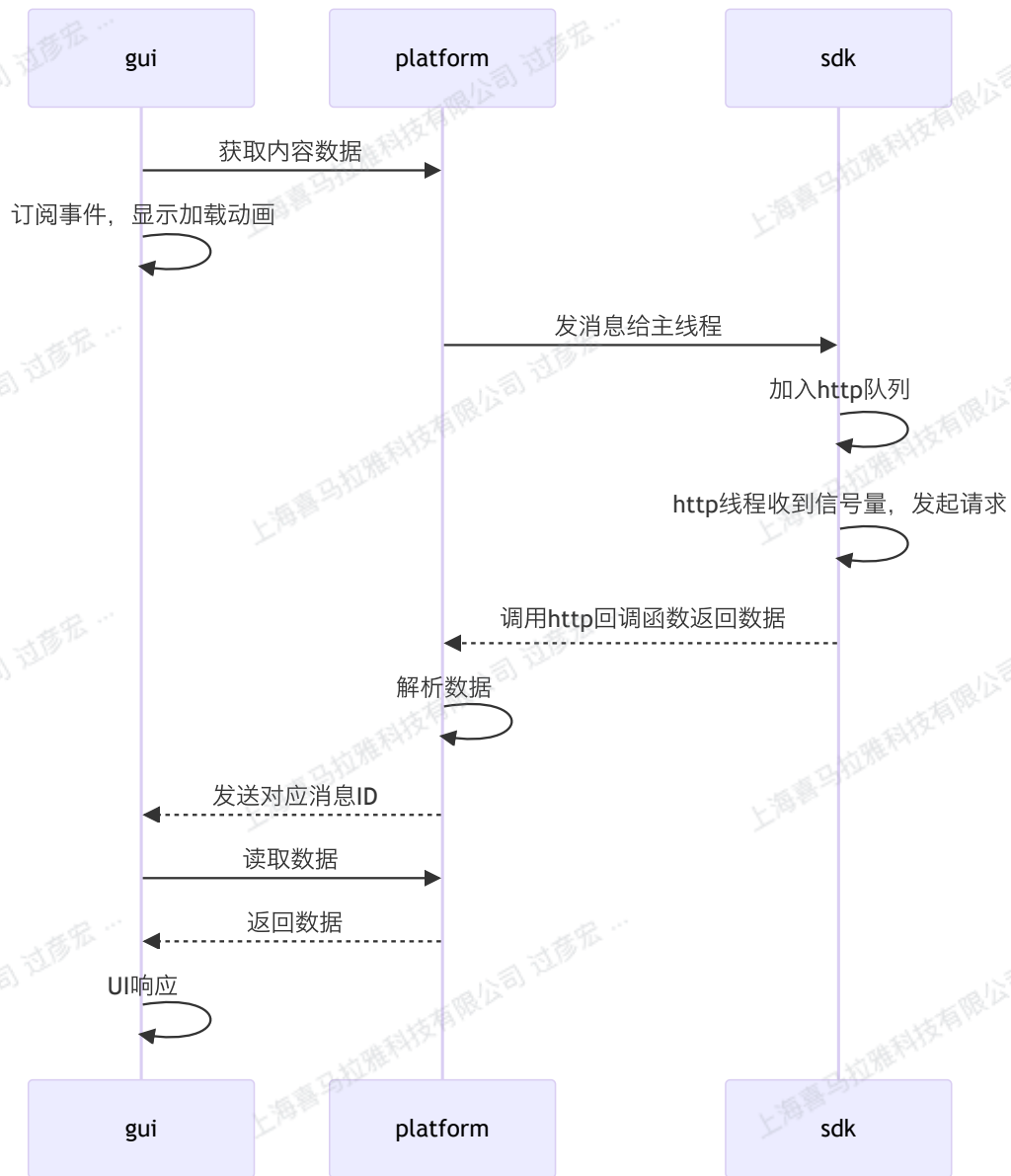
AI功能时序图



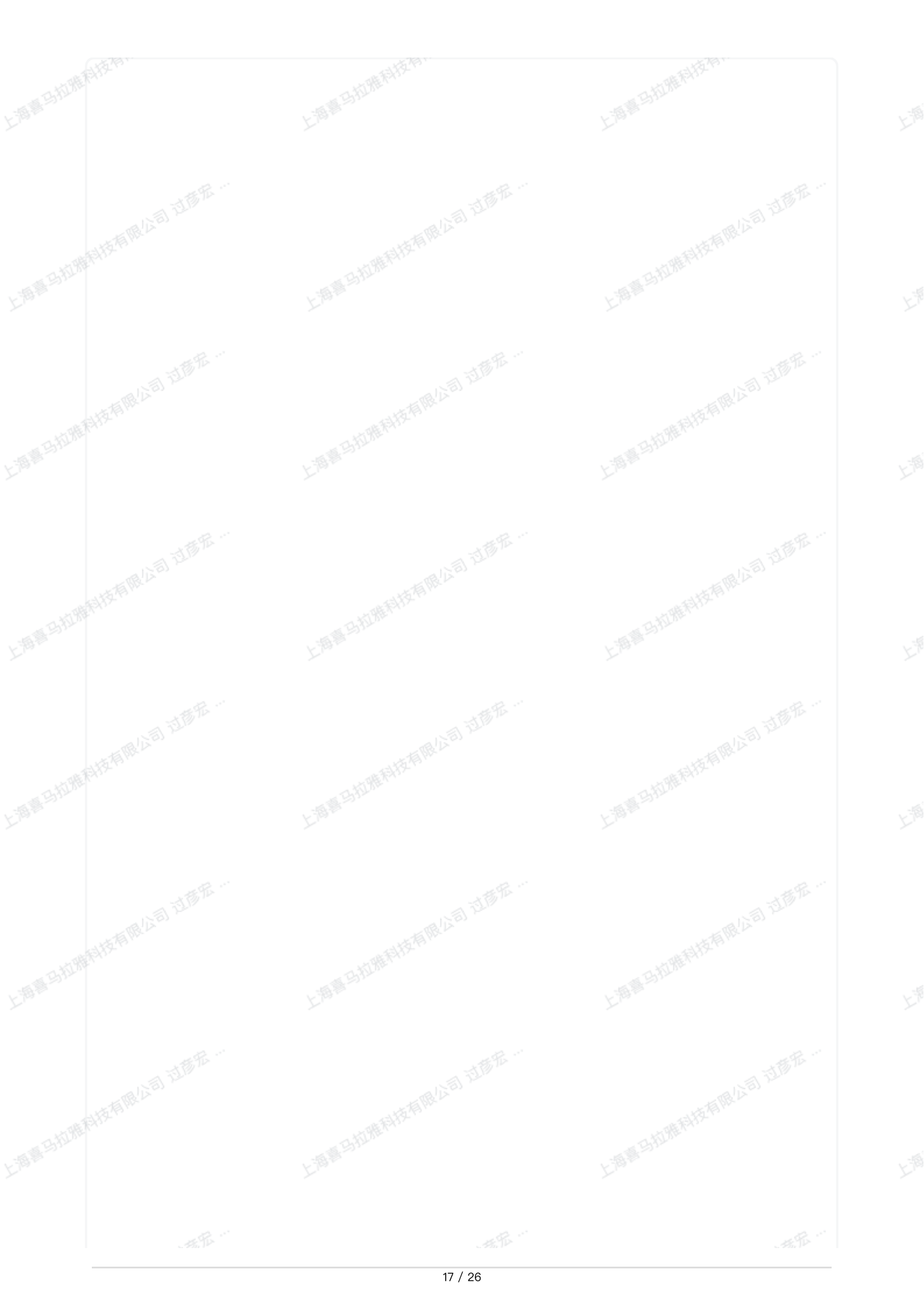


3. HTTP接口请求时序图:

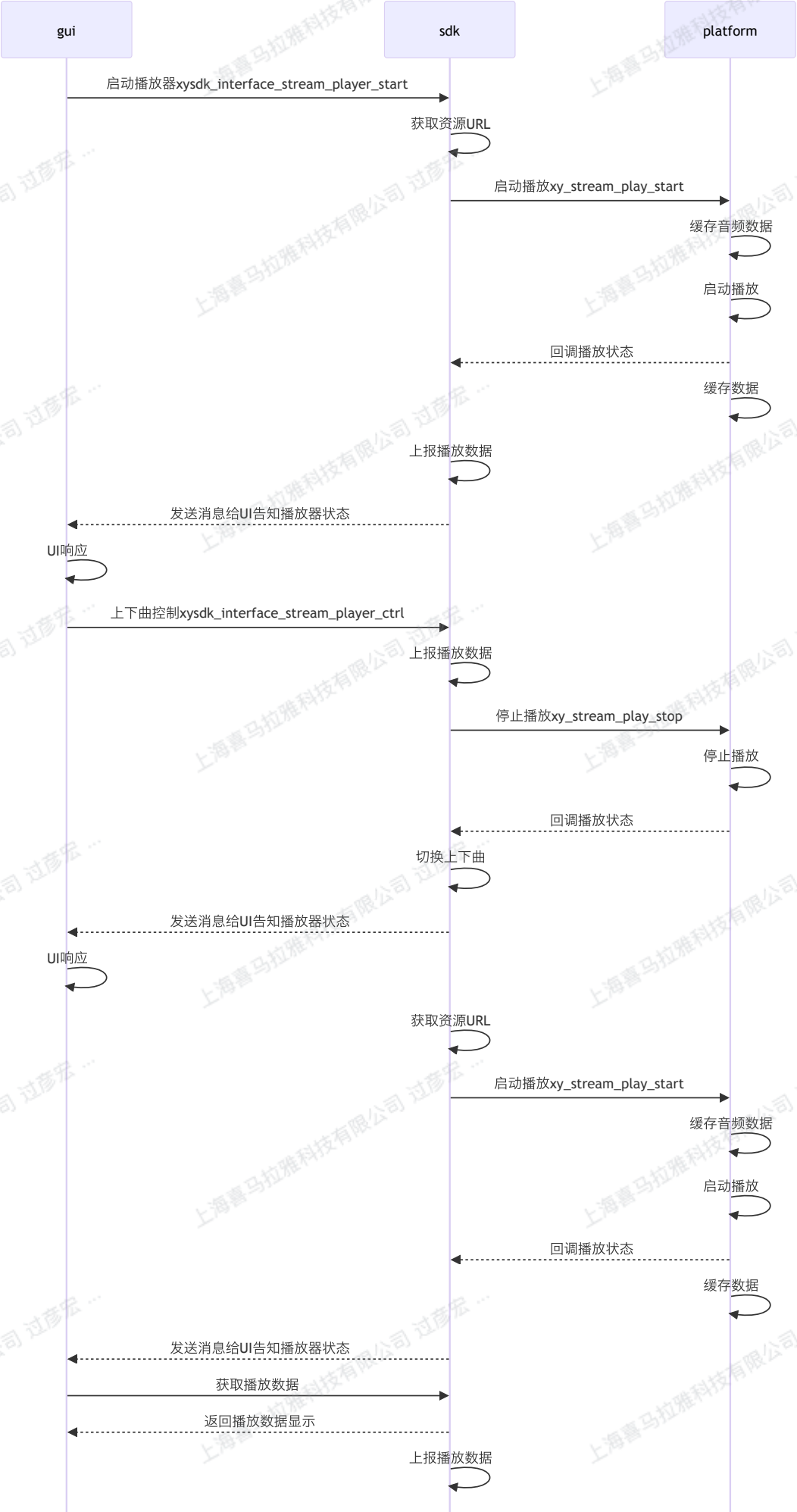
HTTP接口请求时序

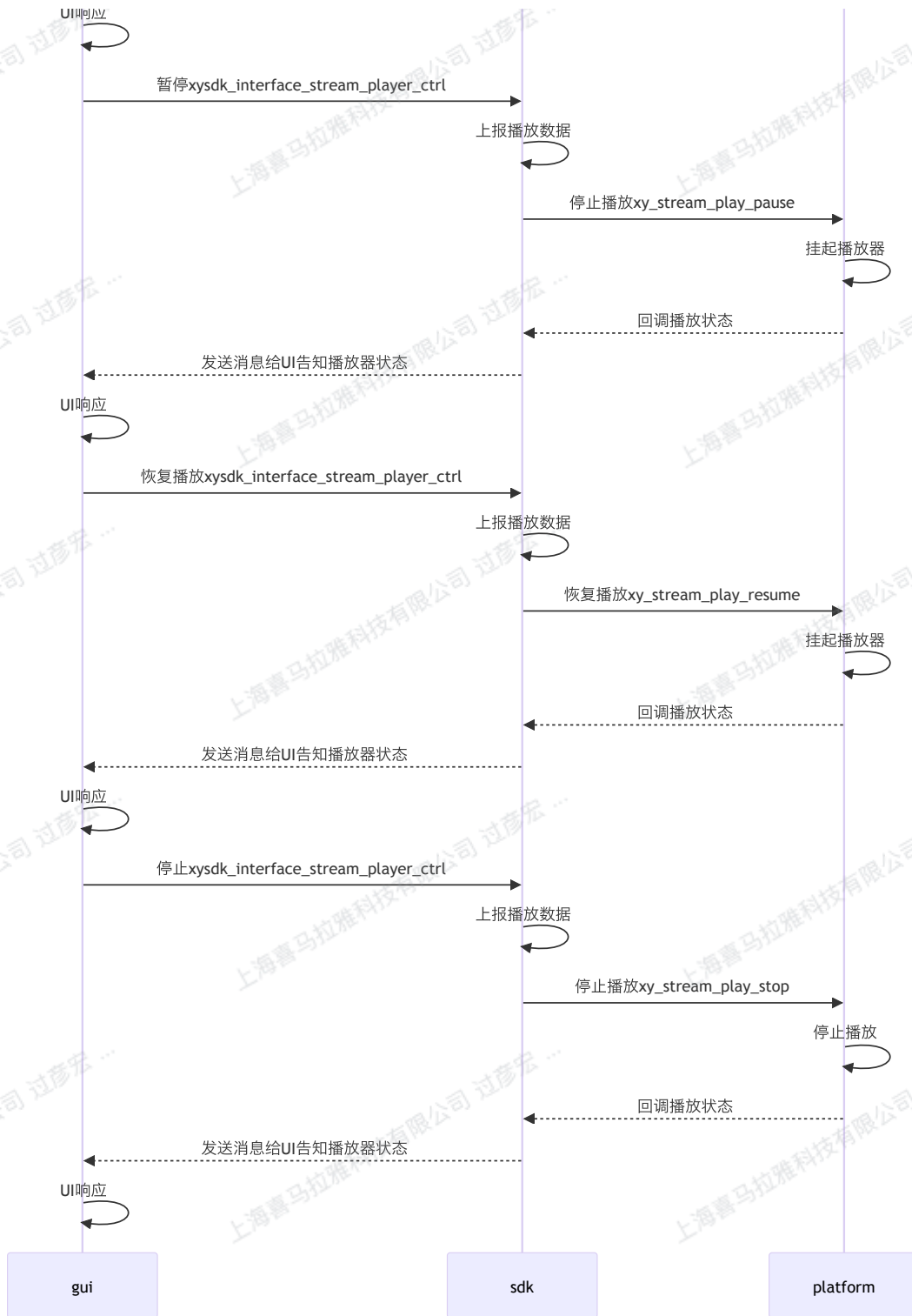


4. 播放器控制时序图



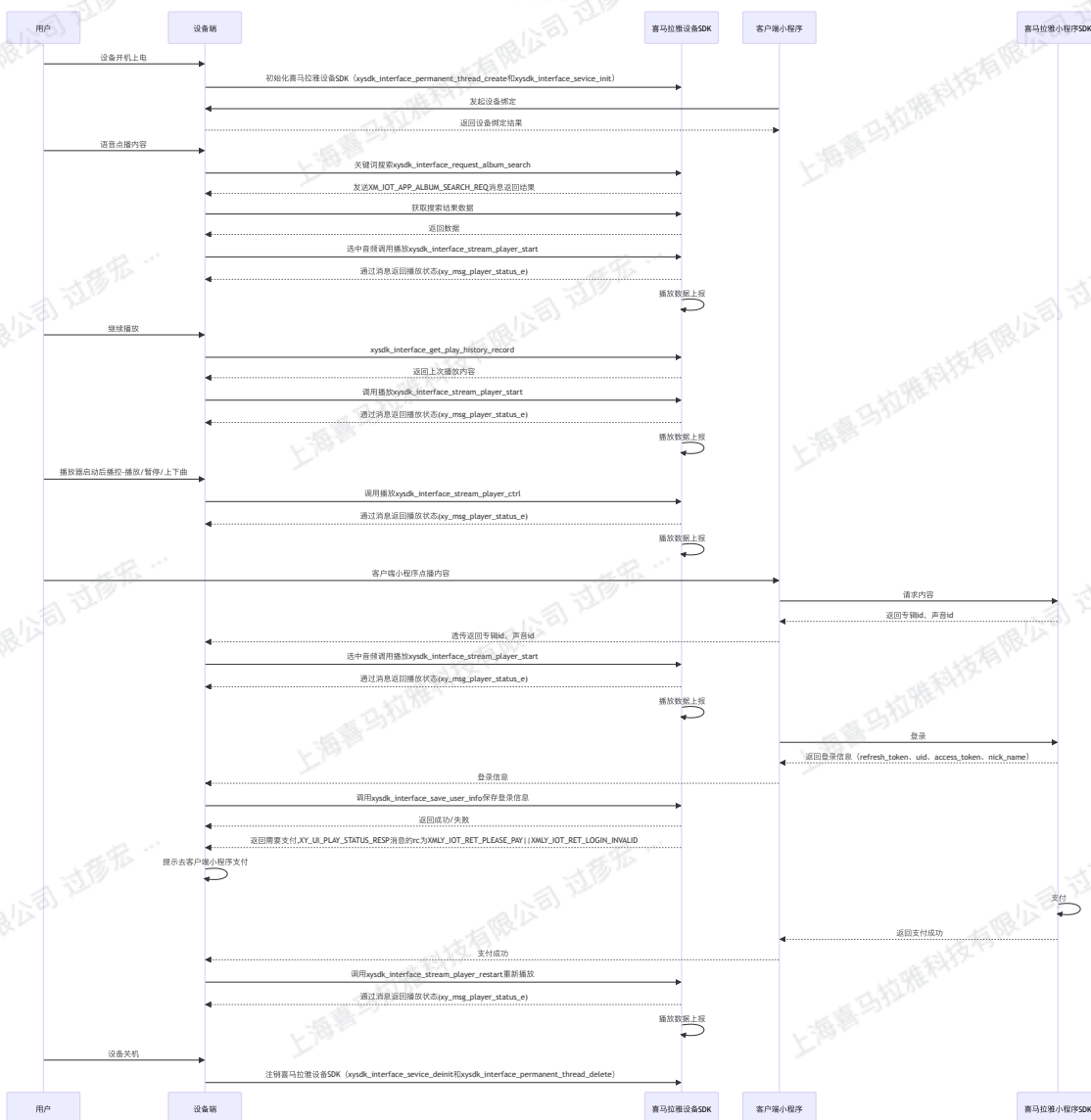
播放控制时序图





5. 无屏方案操作时序图（儿童SDK）

无屏方案操作时序图

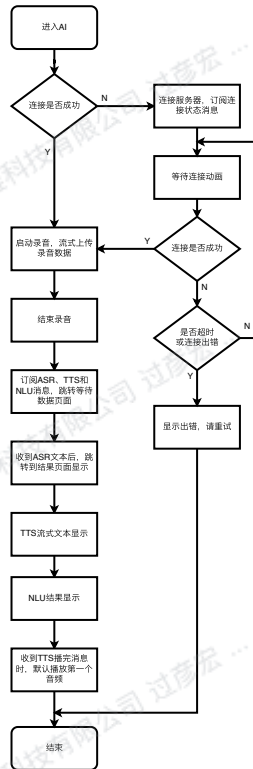


七、功能流程说明

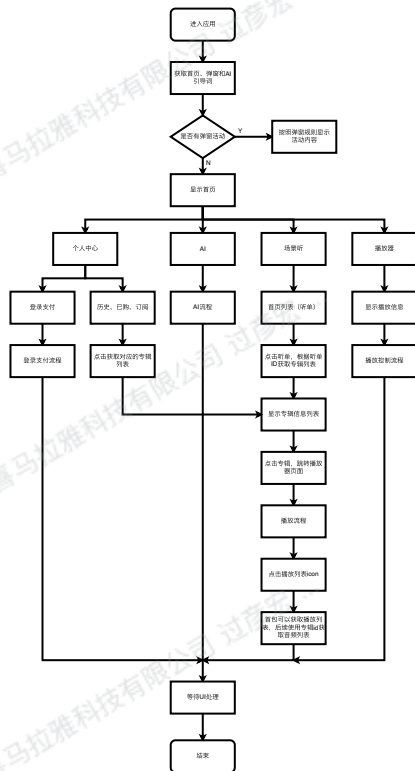
1. 应用流程图



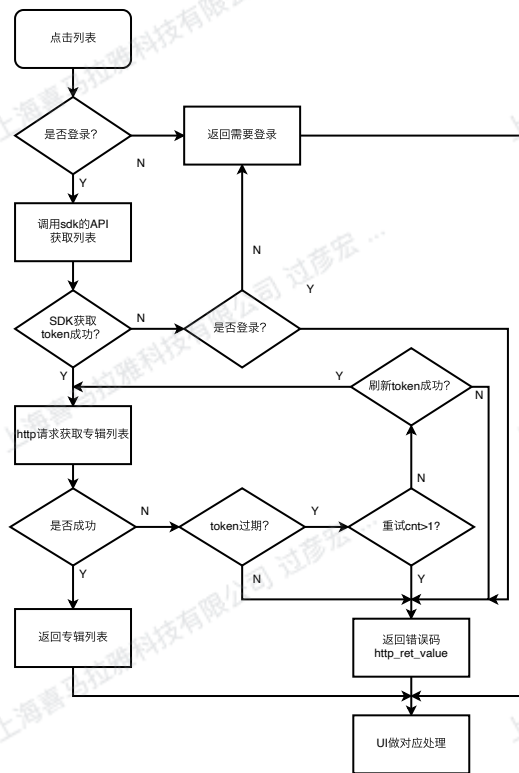
3. AI操作流程



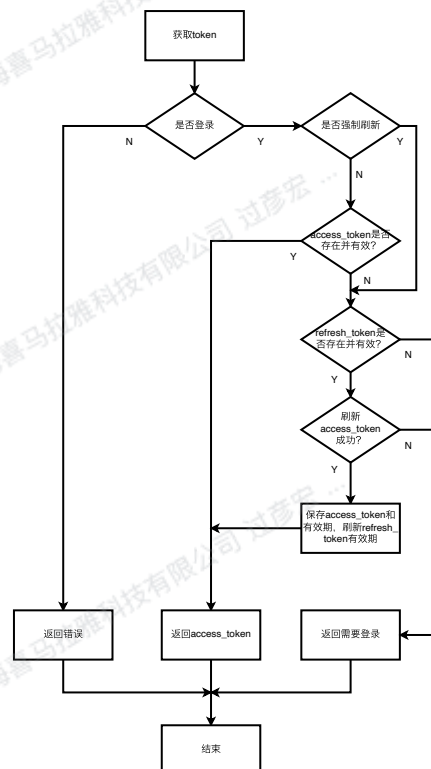
4. 内容获取流程



5. 播放流程图



8. SDK中token维护逻辑



八、付费类型枚举

1. 专辑付费类型-ALBUM_PAY_TYPE

枚举值	说明	专辑列表角标	支付二维码action参数
0	免费	\	\
1	vip	VIP	personal_center
2	单购	付费	buy
3	限免	限免	\
4	vip 单购	VIP	personal_center
5	vip抢先听(免费专辑含vip声音)	\	不支持
6	ximi会员	\	不支持
7	ximi会员 单购	付费	buy
8	儿童会员	儿童vip	personal_center
9	儿童会员 单购	儿童vip	personal_center
10	svip	SVIP	personal_center
11	svip 单购	SVIP	personal_center

2. 音频付费类型-TRACK_PAY_TYPE

枚举值	说明	音频角标
0	免费	\
1	付费	有权益时不显示，没权益时根据ALBUM_PAY_TYPE显示付费角标
2	试听	\

3	vip抢先听	有权益时不显示，没权益时显示锁
4	ximi抢先听	有权益时不显示，没权益时显示锁
5	ximi会员	有权益时不显示，没权益时显示锁